FIAP · 2TDS · NAVEGANDO PELO MUNDO DEVOPS

Projeto - Cidades ESG Inteligentes MoveSaúde API

Pipeline CI/CD, containerização e deploy em staging e produção

Integrante	RM
João Pedro Aranha Moscatello	565854
Lucas Vilarim de Souza	565685

Repositório: https://github.com/4ranh4/movesaude-devops
Staging: https://movesaude-api-staging.onrender.com · Produção: https://movesaude-api-producao.onrender.com

1. O projeto

A **MoveSaúde** conecta pessoas com deficiência, idosos e pacientes em tratamento contínuo a uma rede de prestadores verificados de transporte acessível, assistido e de remoção não emergencial. A API, em **Java 25 com Spring Boot 4.1** e **PostgreSQL**, faz a triagem da modalidade pelas necessidades do passageiro, o matching com até três opções compatíveis, a cotação por trecho e a aplicação dos créditos mensais do convênio.

O tema **Cidades ESG Inteligentes** aparece nos indicadores que a própria API calcula em `GET /api/esg/indicadores`:

Pilar	Indicadores
Ambiental	km rodados, CO_2 emitido e evitado pela frota elétrica ou híbrida, % da frota limpa
Social	corridas acessíveis ou médicas, idas a tratamentos contínuos (hemodiálise, oncologia, terapia), passageiros atendidos, valor subsidiado pelos convênios
Governança	% de prestadores verificados (só eles recebem corridas), comissão única e preço detalhado linha a linha

Ambientes

Ambiente	Execução	Banco	Origem
Local	Docker Compose: API + PostgreSQL 17	container `db`, volume `movesaude-pgdata`	qualquer branch
Staging	Render, container Docker (plano free)	Supabase PostgreSQL 17, schema e usuário `movesaude_staging`	branch `develop`
Produção	Render, container Docker (plano free)	Supabase PostgreSQL 17, schema e usuário `movesaude_producao`	branch `main`

Staging e produção ficam no mesmo projeto Supabase, mas isolados: cada ambiente tem o próprio usuário de banco, dono apenas do próprio schema e sem permissão no schema do outro.

2. Pipeline CI/CD

Ferramenta: GitHub Actions (`.github/workflows/ci-cd.yml`), com GitHub Environments (`staging` e `production`) e GitHub Container Registry. **Destino do deploy:** Render, com os dois serviços descritos como código no Blueprint `render.yaml` .

1 · Build e testes
JDK 25 + cache Maven, `mvn verify` : compila, roda os 37 testes e gera o .jar

→

2 · Imagem Docker
build multi-stage, sobe API + PostgreSQL com Compose, smoke test, push no GHCR

→

3a · Deploy staging
push em `develop` : deploy hook, espera o commit no ar e roda smoke test com escrita

/

3b · Deploy produção
push em `main` : aprovação do environment, deploy hook, espera o commit e smoke test somente leitura

Etapas e lógica

Job	Gatilho	O que faz
Build e testes automatizados	todo push e PR em `develop` / `main`	`actions/setup-java` (Temurin 25) com cache do Maven; `./mvnw -B verify` ; publica relatórios de teste e o .jar como artefatos; escreve a tabela de testes no resumo da execução.
Imagem Docker e teste com Docker Compose	após o job 1	`docker build --build-arg GIT_SHA=…` ; `docker compose up` com API + PostgreSQL; espera o health check `healthy` ; roda `scripts/smoke-test.sh` com um fluxo completo de corrida; em pushes, publica no GHCR com as tags `sha-<commit>` , `<branch>` e `staging` / `production` .
Deploy em staging	push em `develop`	Aciona o deploy hook do serviço de staging (secret `RENDER_DEPLOY_HOOK_STAGING`); `scripts/wait-for-deploy.sh` espera `/api/info` responder com o commit testado; smoke test em staging, criando e confirmando uma corrida.
Deploy em produção	push em `main`	Environment `production` com revisão obrigatória; deploy hook de produção; espera o commit no ar; smoke test somente leitura.

- **Promoção:** o código é integrado em `develop` e vai automaticamente para staging. Validado, um pull request `develop → main` leva a mesma versão para produção.
- **Nada vai para um ambiente sem passar nos testes:** o deploy automático do Render está desligado (`autoDeployTrigger: off`), e só o pipeline dispara o deploy.
- **Deploy confirmado de verdade:** o pipeline só termina quando a instância nova responde com o commit esperado e passa no smoke test.
- **Segredos fora do código:** deploy hooks ficam nos environments do GitHub, e as credenciais do banco nas variáveis do Render.

3. Docker

Arquitetura

Rede movesaude-backend
bridge interna entre os serviços

api · movesaude-api
imagem do Dockerfile, porta 8080 publicada, health check `/actuator/health`

→

db · postgres:17-alpine
acessado pelo nome `db`, porta não exposta, `pg_isready`

Volume movesaude-pgdata
dados persistentes do PostgreSQL

A API só sobe depois que o banco responde (`depends_on: condition: service_healthy`). As variáveis vêm do `.env`, cujo modelo é o `.env.example`. O Flyway cria o schema e as tabelas e carrega os dados de demonstração na primeira inicialização.

Imagem criada

Item	Valor
Nome	`ghcr.io/4ranh4/movesaude-api`
Tags	`sha-<commit>`, `develop`, `main`, `staging`, `production`
Base de build / runtime	`eclipse-temurin:25-jdk` → `eclipse-temurin:25-jre`
Tamanho	173 MB compactada no registry (9 camadas); o .jar tem 55 MB
Usuário	`movesaude`, sem privilégios de root
Health check	`curl /actuator/health/liveness` a cada 15 s

Estratégias

- **Multi-stage:** o JDK e o Maven ficam na etapa de build; a imagem final leva só o JRE e o .jar.
- **Cache de camadas:** o `pom.xml` e as dependências vêm antes do código, então uma mudança no código não baixa as dependências de novo.
- **Mesma imagem em todos os ambientes:** só mudam as variáveis `APP_ENV`, `DB_URL`, `DB_USER`, `DB_PASSWORD`, `DB_SCHEMA` e `PORT`.
- **Rastreabilidade:** `GIT_SHA` entra na imagem e aparece em `/api/info`.
- **JVM para containers pequenos:** `MaxRAMPercentage=60`, `SerialGC` e `TieredStopAtLevel=1`, para caber nos 512 MB do plano gratuito.

Comandos usados

```
cp .env.example .env
docker compose up -d --build          # constrói a imagem e sobe API + PostgreSQL
docker compose ps                     # estado e health dos serviços
docker compose logs -f api            # logs da API
./scripts/smoke-test.sh http://localhost:8080 local --fluxo
docker compose down -v                # derruba e apaga o volume

# no pipeline
```

```
docker build --build-arg GIT_SHA=$GITHUB_SHA -t movesaude-api:ci .
IMAGE_TAG=ci docker compose up -d --no-build
docker tag movesaude-api:ci ghcr.io/4ranh4/movesaude-api:sha-<commit>
docker push ghcr.io/4ranh4/movesaude-api:sha-<commit>
```

4. Prints do pipeline rodando

Execuções: https://github.com/4ranh4/movesaude-devops/actions

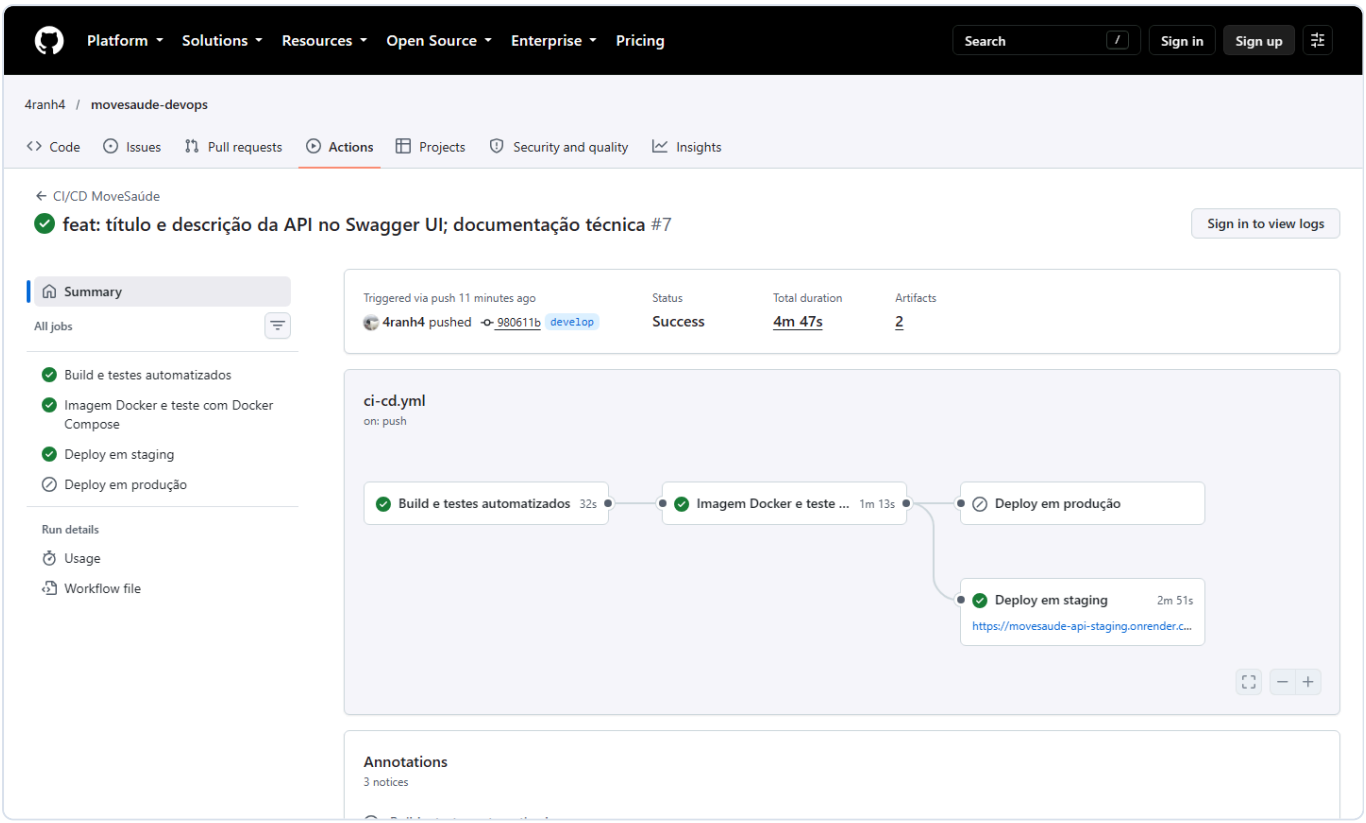


Figura 1. Push em `develop`: build e testes → imagem Docker com Compose → deploy em staging.

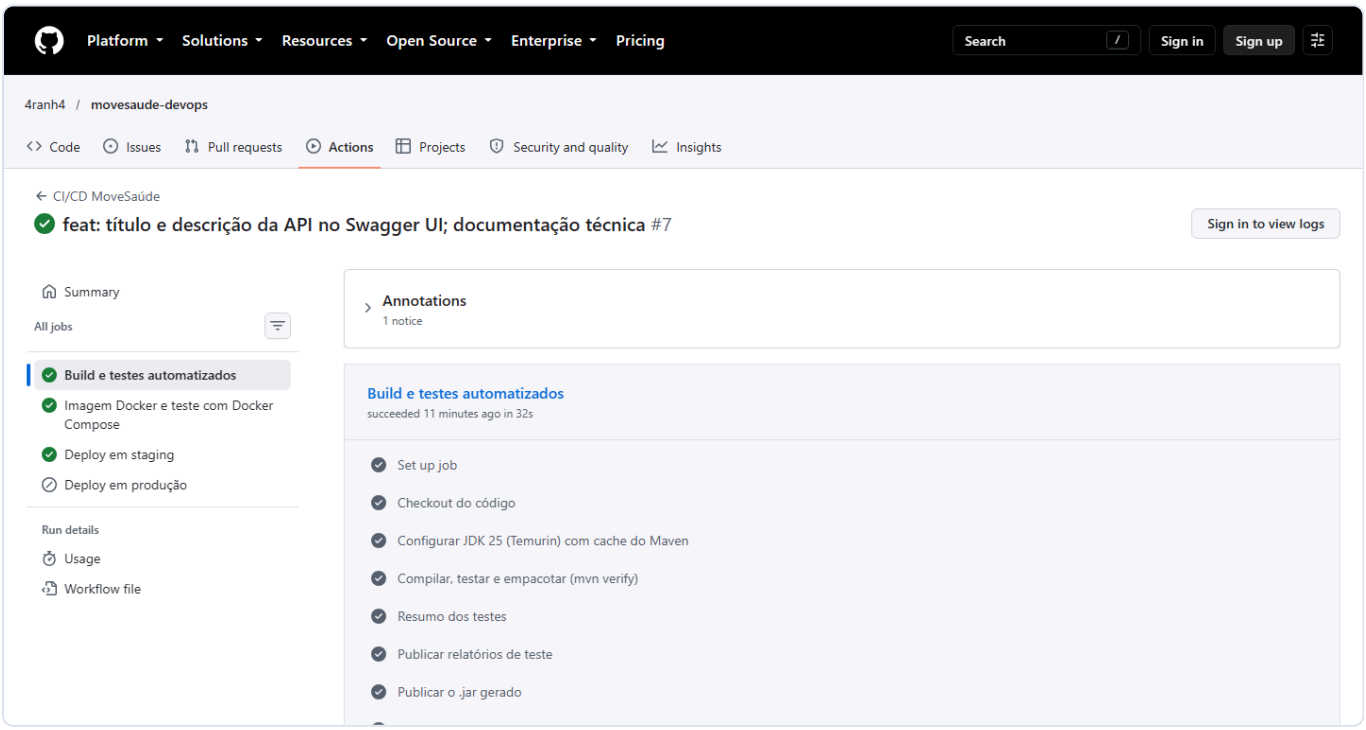


Figura 2. Job de build e testes: JDK 25, `mvn verify` (compila, testa e empacota), relatórios e .jar publicados como artefatos.

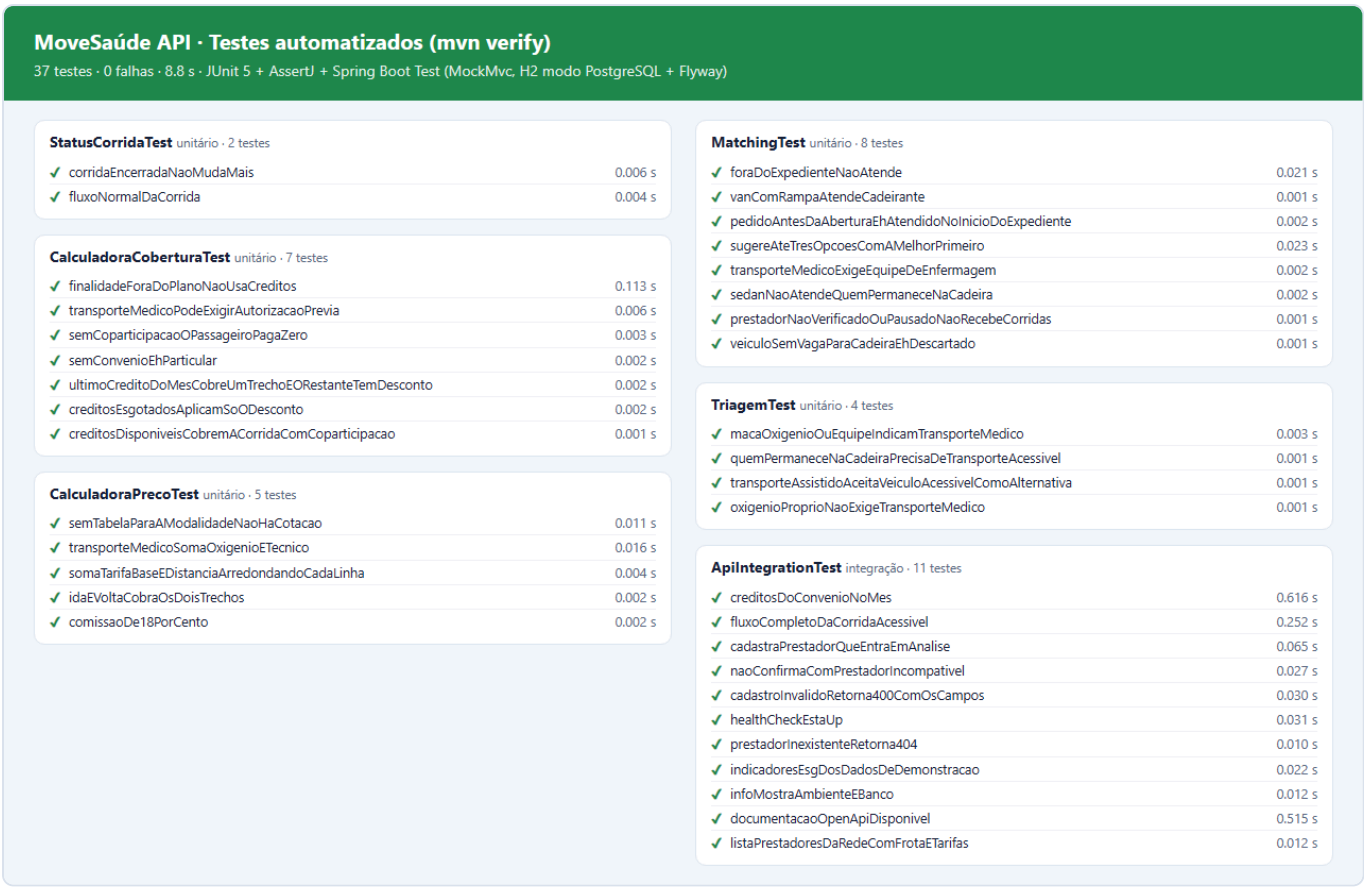


Figura 3. Suíte de testes executada pelo `mvn verify`: 37 testes, 0 falhas (26 unitários e 11 de integração).

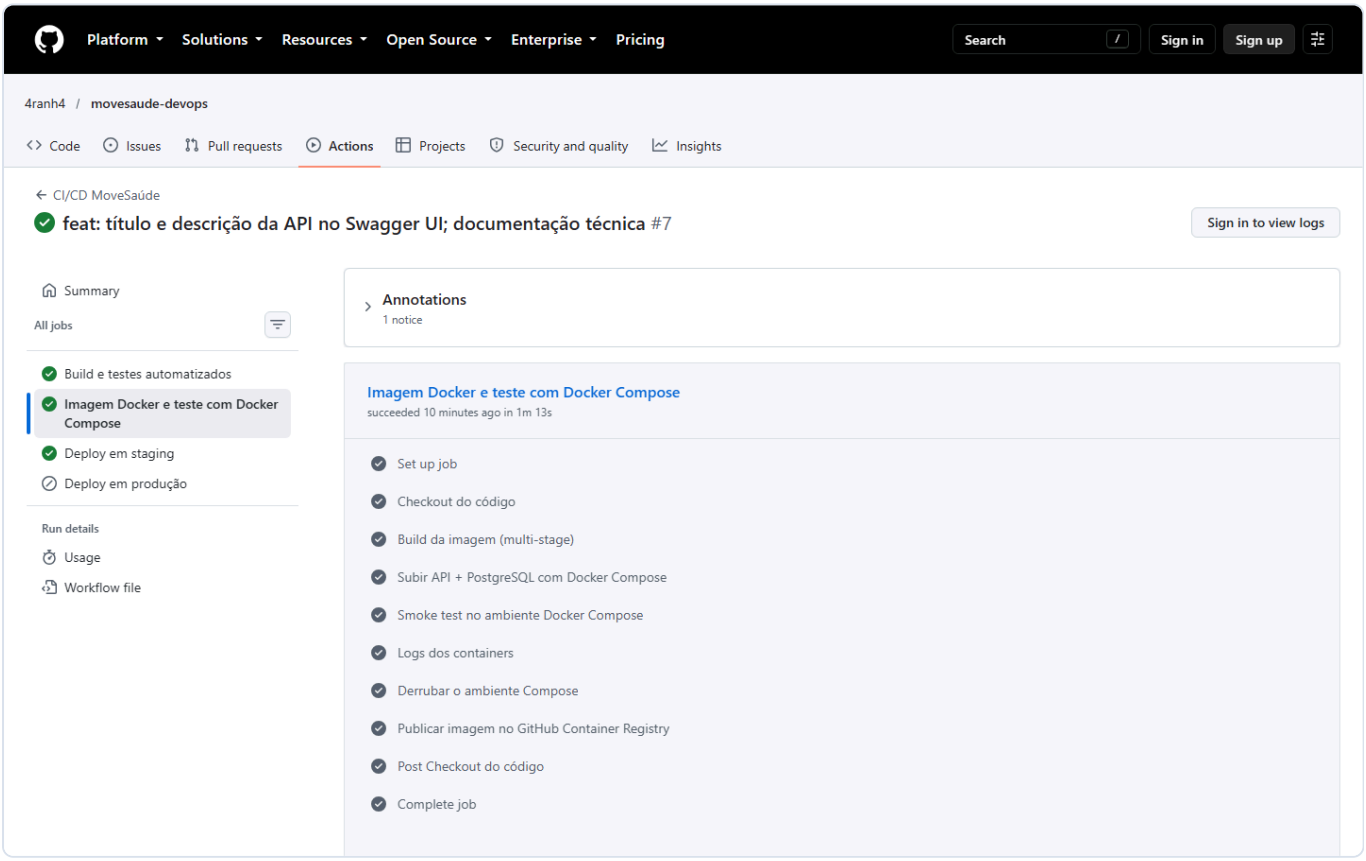


Figura 4. Etapas do job Docker: build multi-stage, subida de API + PostgreSQL com Compose, smoke test e publicação no GHCR.

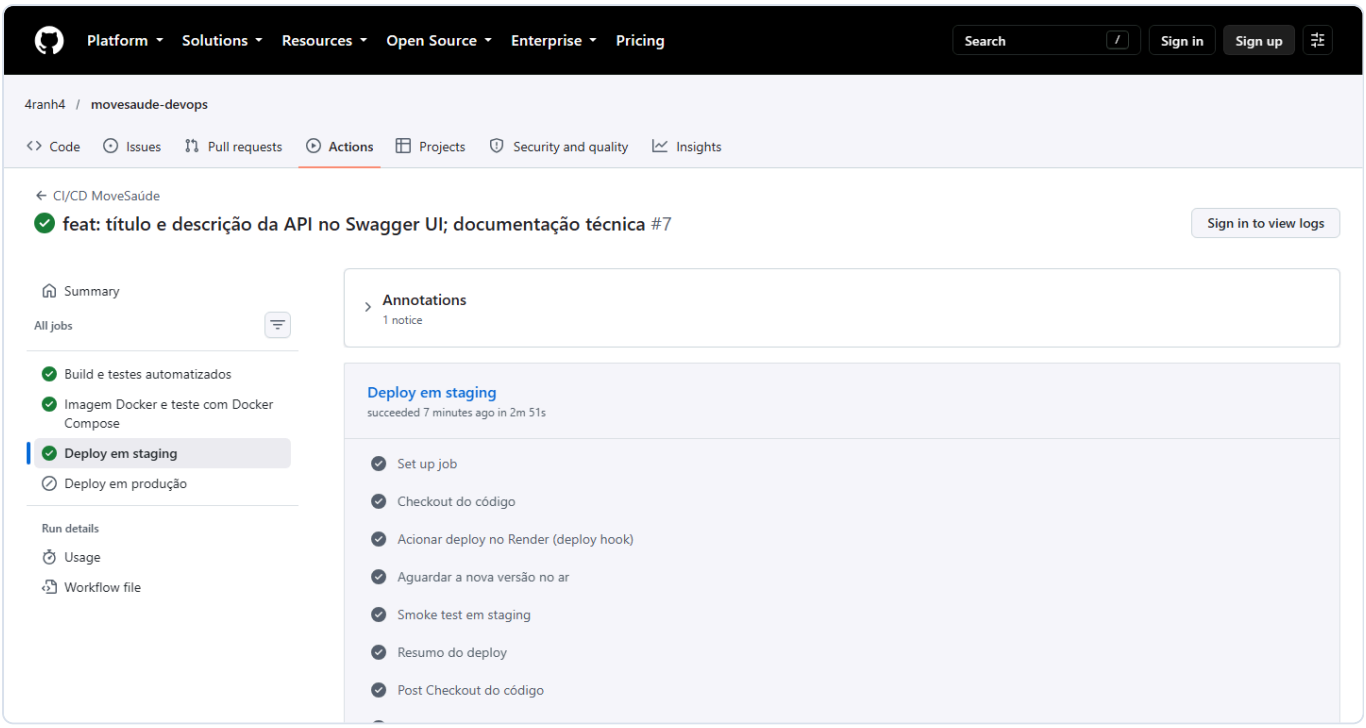


Figura 5. Deploy em staging: deploy hook do Render, espera do commit no ar e smoke test.

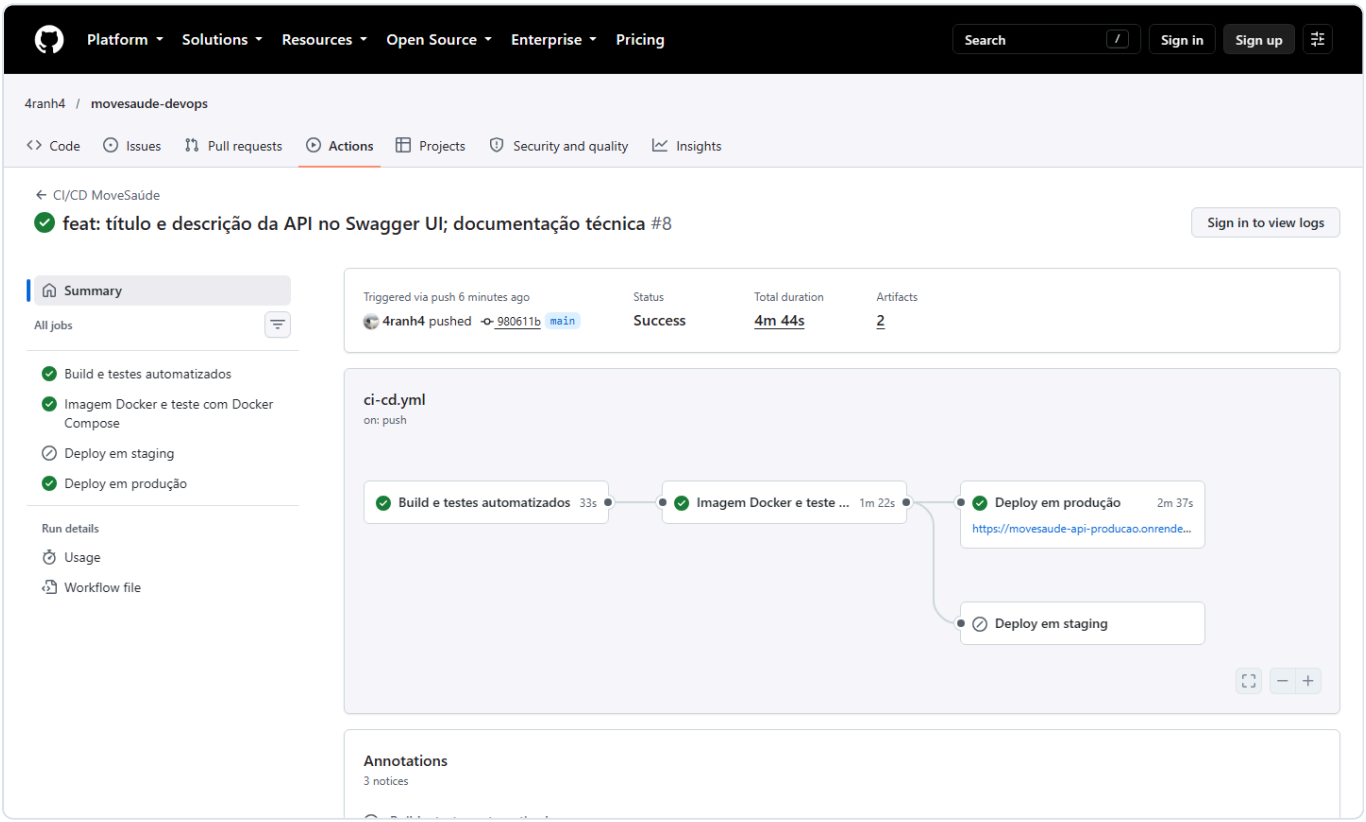


Figura 6. Push em `main`: mesma esteira, com deploy no ambiente de produção.

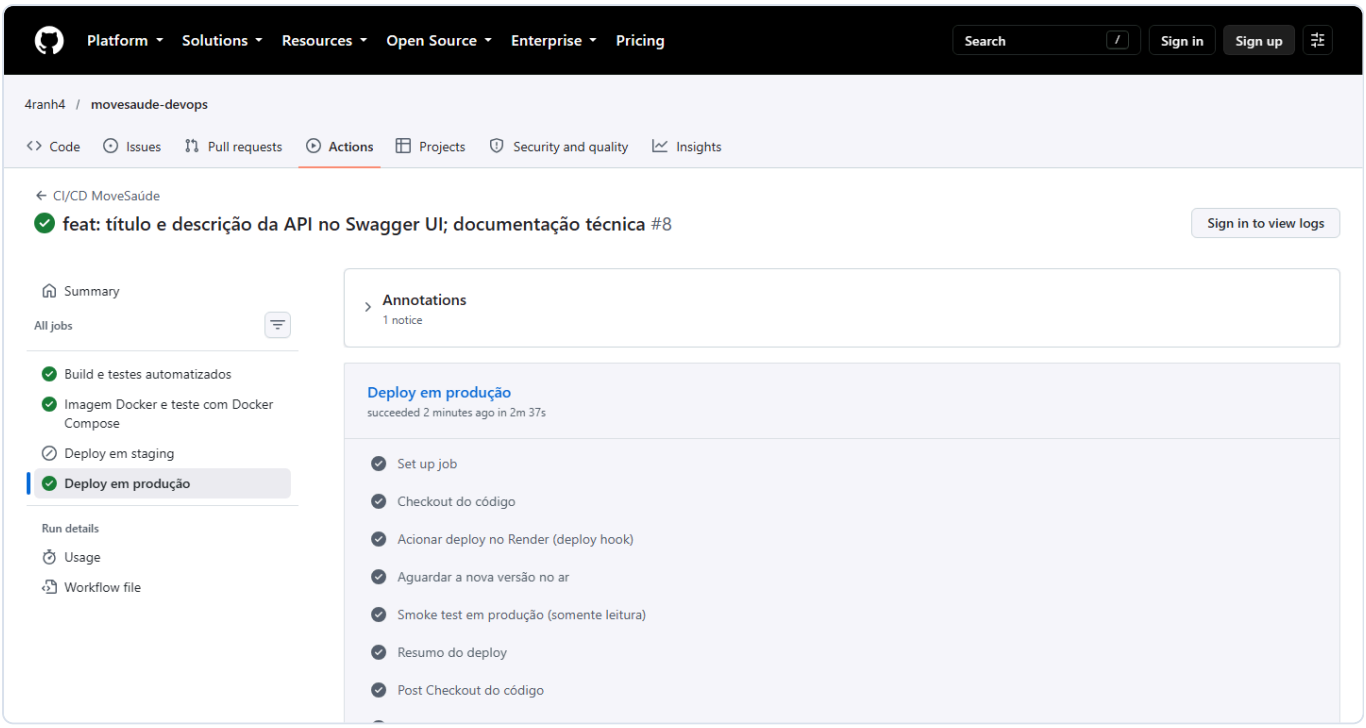


Figura 7. Deploy em produção: deploy hook, confirmação do commit e smoke test somente leitura.

5. Ambientes staging e produção funcionando

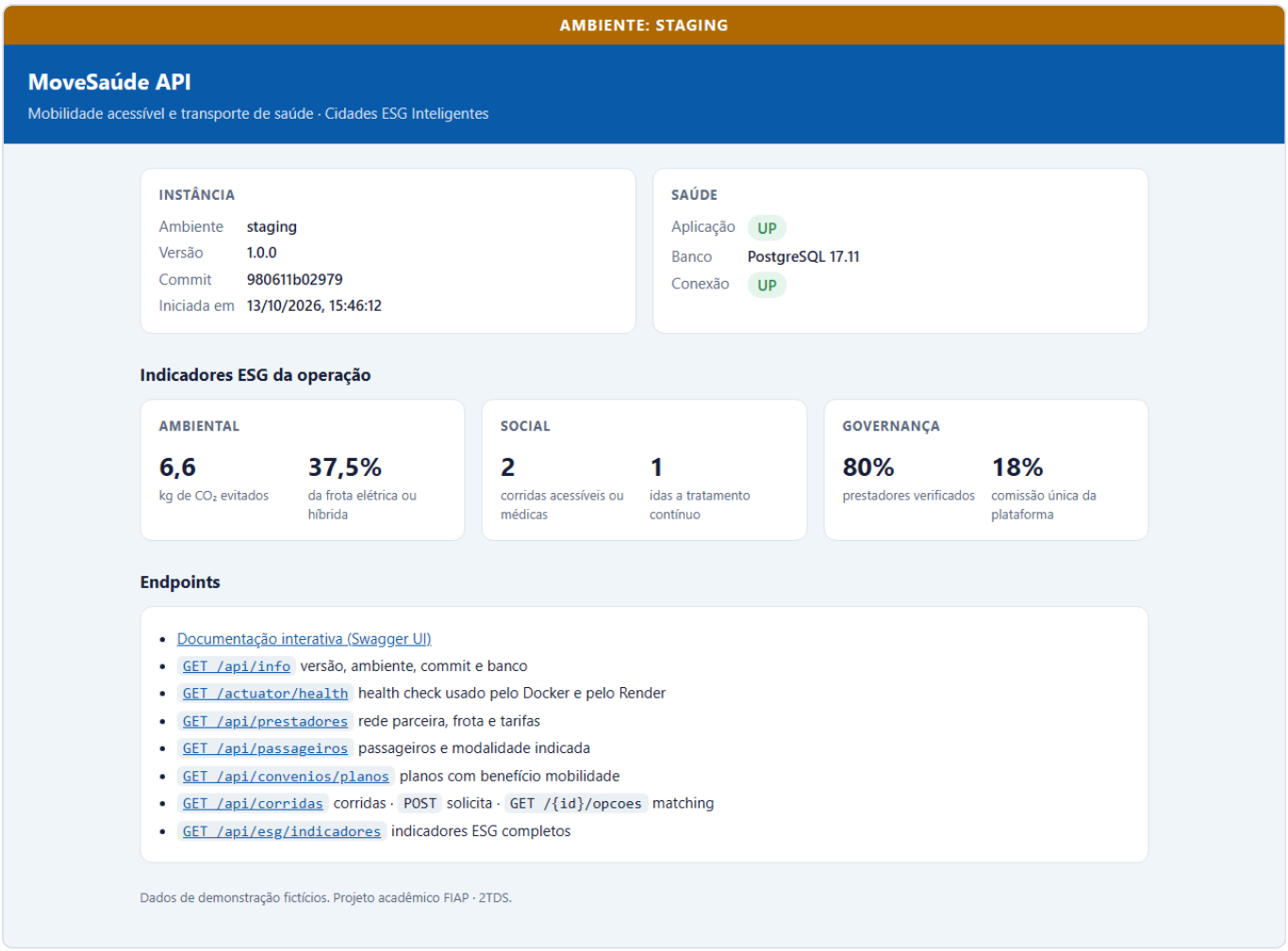


Figura 8. Staging (https://movesaude-api-staging.onrender.com): ambiente, commit, PostgreSQL do Supabase e indicadores ESG.

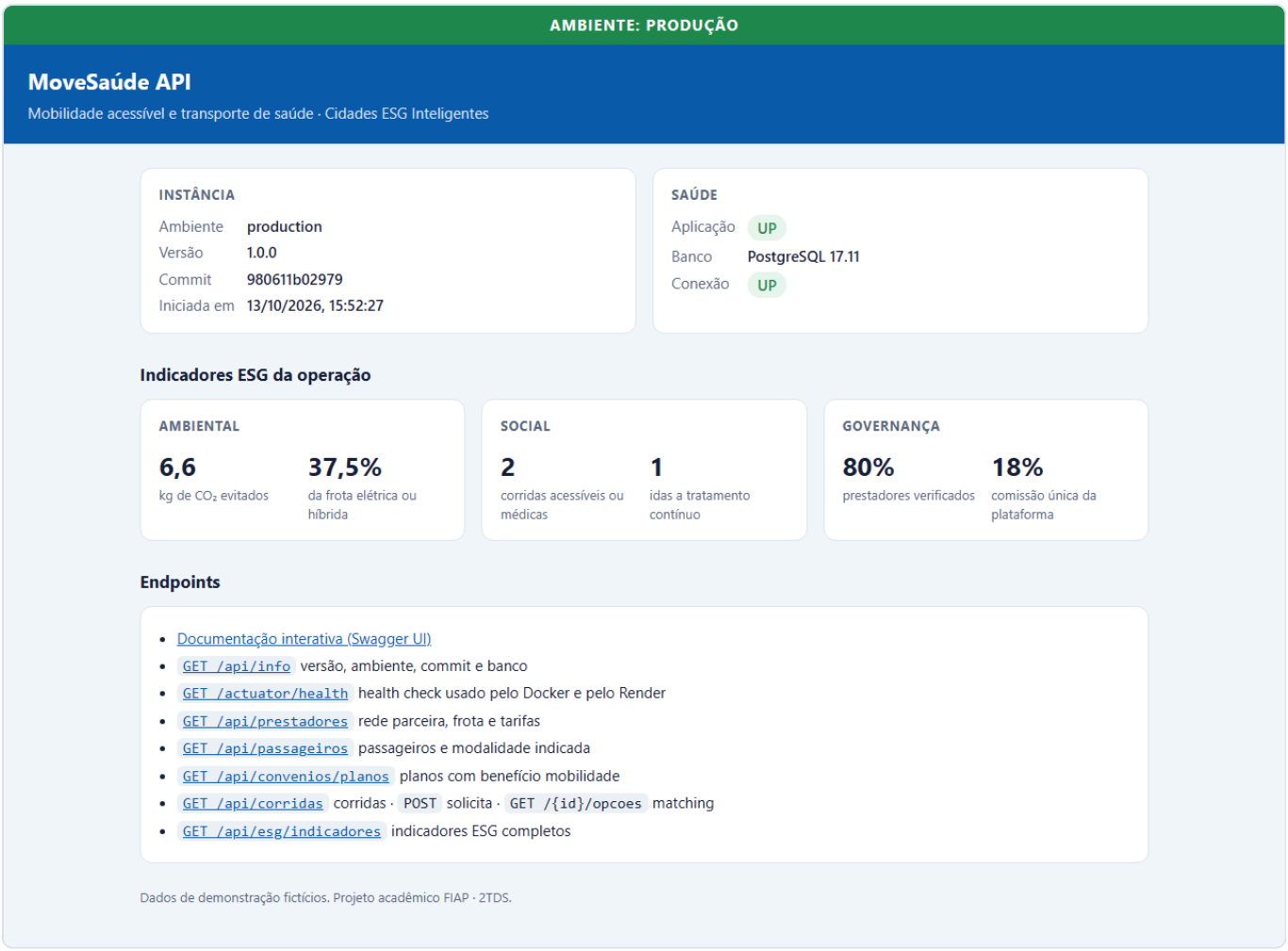


Figura 9. Produção (https://movesaude-api-producao.onrender.com).

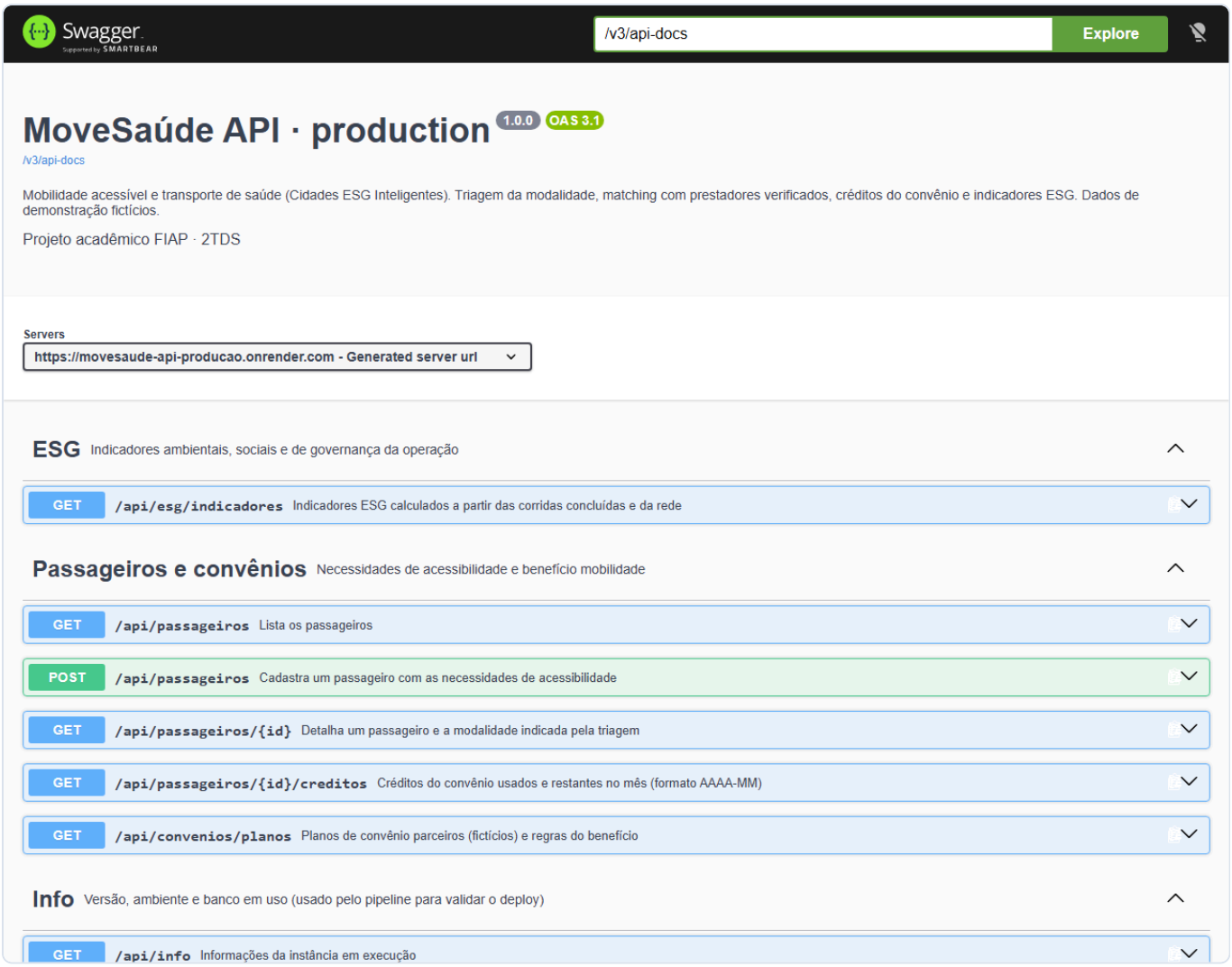


Figura 10. Swagger UI da API em produção.

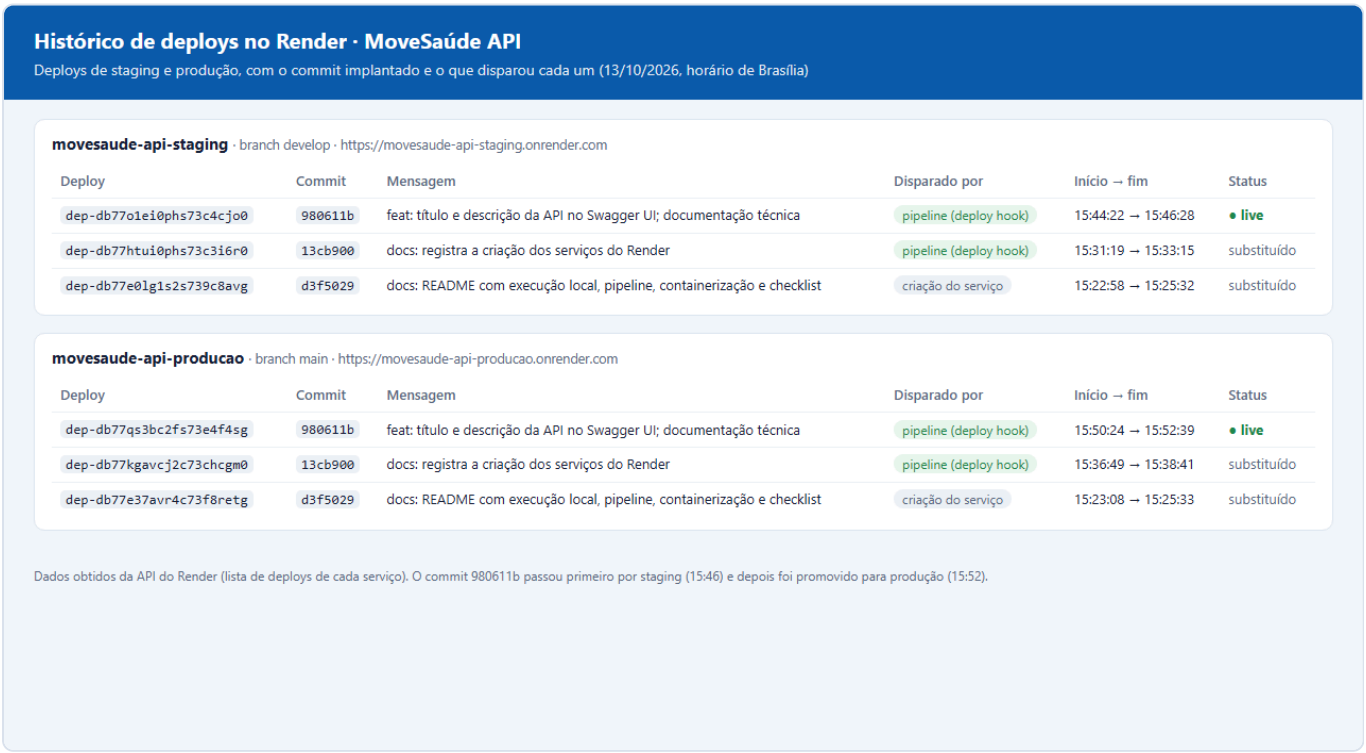


Figura 11. Histórico de deploys no Render: o commit 980611b foi para staging e depois promovido para produção, disparado pelo pipeline (deploy hook).

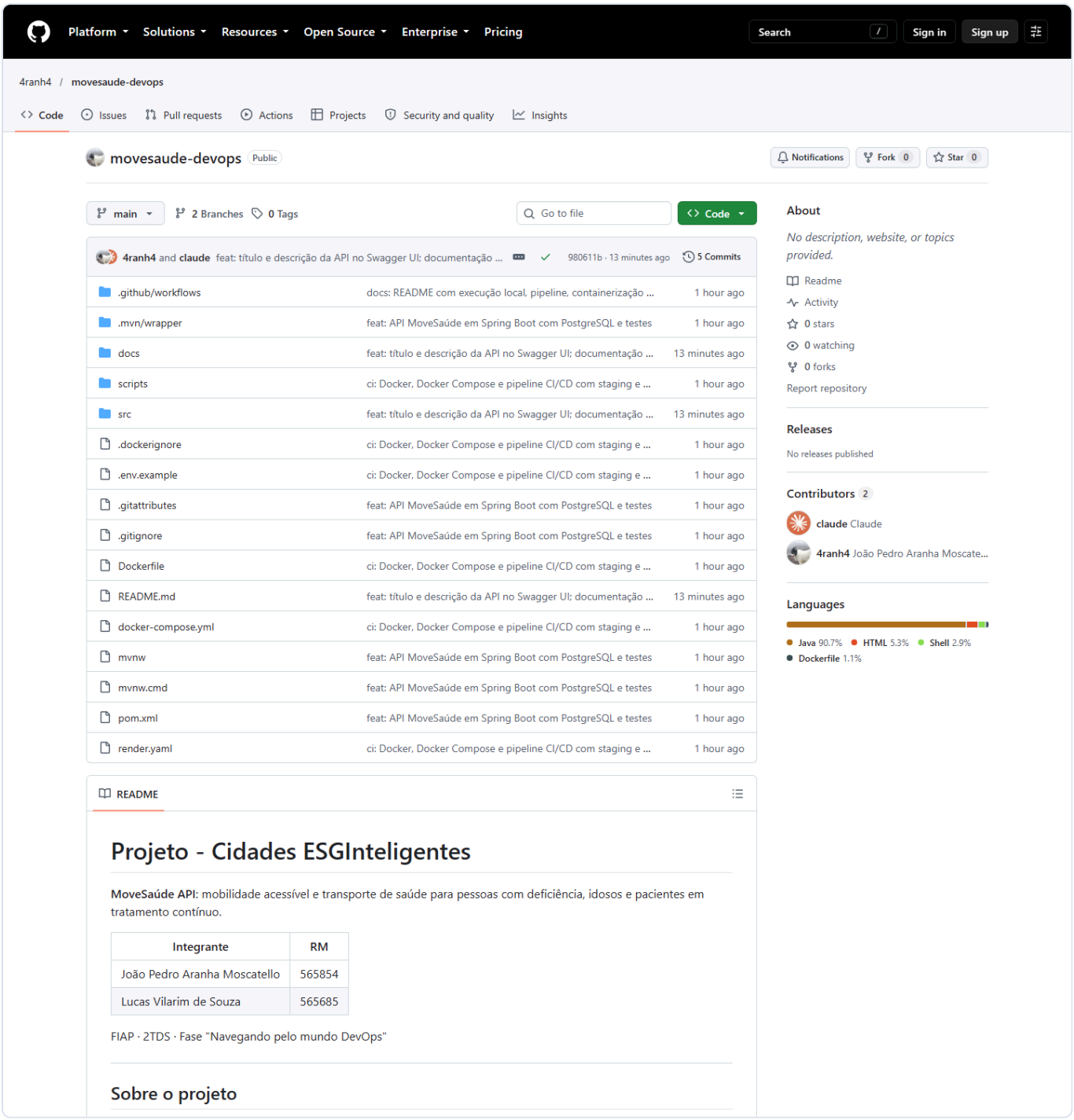


Figura 12. Repositório público no GitHub com o código, o pipeline e o README.

6. Desafios encontrados e como foram resolvidos

Desafio	Solução
O protótipo do MoveSaúde era só front-end (React), e a atividade pede Java Spring ou C#.	Criamos a API em Spring Boot 4.1 e levamos para Java as regras do protótipo (triagem, cotação, matching e créditos do convênio), cobertas por 37 testes.
O plano gratuito do Supabase permite só 2 projetos ativos, e um já estava em uso.	Staging e produção ficaram no mesmo projeto, isolados por **schema e usuário de banco**: cada usuário é dono só do próprio schema.
A conexão direta do Supabase é só IPv6, e o Render e o GitHub Actions usam IPv4.	Conexão pelo **pooler Supavisor em modo sessão** (compatível com prepared statements do JDBC). Testando, descobrimos que o host certo do projeto é `aws-0-us-east-1`; o `aws-1` respondia "tenant not found".
Criar as senhas do banco sem expor o texto puro.	As senhas foram geradas localmente, e só o hash **SCRAM-SHA-256** foi enviado ao Supabase. O texto puro fica apenas nas variáveis secretas do Render.
O Jackson 3, padrão do Spring Boot 4, recusava JSON sem campos booleanos opcionais (erro 400).	Um teste de integração pegou o erro. Configuramos `spring.jackson.deserialization.fail-on-null-for-primitives=false`.
O deploy hook do Render é assíncrono: ele aceita o pedido antes de a versão nova estar no ar.	O `wait-for-deploy.sh` compara o commit de `/api/info` com o SHA do pipeline. Só depois disso roda o smoke test.
O smoke test quebrou porque o JSON do Actuator vem em ordem alfabética.	O script passou a consultar endpoints específicos (`/actuator/health/db` e `/readiness`), em vez de depender da ordem dos campos.
O plano gratuito do Render tem 512 MB de RAM e hiberna o serviço sem uso.	JVM ajustada para containers, timeouts generosos e novas tentativas (retries) nos scripts. A primeira requisição depois da hibernação acorda o serviço.
Testar sem depender de Docker na máquina de desenvolvimento.	Os testes usam H2 no modo PostgreSQL com as mesmas migrations. O PostgreSQL real é validado no job do Docker Compose e no Supabase.

7. Checklist de entrega

Item	OK
Projeto compactado em .ZIP com estrutura organizada	☑ OK
Dockerfile funcional	☑ OK
docker-compose.yml ou arquivos Kubernetes	☑ OK
Pipeline com etapas de build, teste e deploy	☑ OK
README.md com instruções e prints	☑ OK
Documentação técnica com evidências (PDF ou PPT)	☑ OK
Deploy realizado nos ambientes staging e produção	☑ OK